

通用运动控制器API接口

- Author: General Test System Inc
- Version: 1.0.0
- Create Date: 2023.5.31
- Update Date: 2023.7.12
- Update Date: 2024.04.22
- Update Date: 2024.04.28
- Update Date: 2024.09.04

1. 引言

目前公司各产线各项目中使用的运动控制器硬件，每一个不同的型号，对于上位机软件都需要使用对应的API DLL，实际上实现的功能都具有一定的共同点，同时，部分运动控制器设备存在独有的初始化逻辑并保留较多经验数值的延迟。
此文档用于统一GTS自研运动控制器设备的控制调用，如：**转台、滑轨、摇摆臂、极轴、搅拌器**等设备，统一明确运动控制器设备所处的阶段及处于该阶段下的状态，可缩短上位机对新增的运动控制器硬件设备的调试时间，新增硬件后，正确配置硬件的情况下，可直接使用API进行功能调试。

本文档只做定义上位机在调用多轴运动控制器时的接口。为了给日后统一做铺垫，在新集成的多种运动控制器中，各个项目组都继承该接口来实现。本接口定义在 Platform.Core 中。

2. 全局定义

2.1. 运动单位

如无特殊说明，本文中所有的 `Unit` 为该轴运动的实际单位，由实际测试过程中的控制需求来确定。

2.2. 轴序号

如无特殊说明，本文中所有的 `dimension/dimensions` 为轴序号/轴序号列表。

用于指定运动控制器可控制的轴，类型 `int`，范围为 `[0, n)`，其中 `n` 为 最大可控制的轴数量

2.3. 错误码

调用函数时，当函数内部或者调用发生错误，均抛出 `GTSDeviceException`。

以下对 exception 做定义

```
1 // deviceName 为设备名称，code 为自定义的错误码，message 为硬件驱动抛出的错误
2 GTSDeviceException(string deviceName, ErrorCode code,string message);
3
4 GTSDeviceException(string deviceName, ErrorCode code,string message, Exception InnerExcpetion);
```

设备名称从异常设备的 `IDevice.Name` 中获取，用于辅助流程捕获时明确出错设备的名称，并提示用户。

错误码详细定义如下：

```
1 public enum ErrorCode
2 {
3     // 参数错误
4     ARGUMENT_ERROR = 10001, // 参数错误
5
6     // 连接错误
7     CONNECT_DISCONNECTED = 20002, // 当前未连接
8     CONNECT_TIMEOUT = 20005, // 连接超时
9
10    // 硬件错误
11    DEVICE_NOT_SUPPORTED = 30001, // 硬件不支持
12    DEVICE_NOT_INITIALIZED = 30002, // 硬件没有初始化
13    DEVICE_BUSY = 30003, // 硬件忙
14    DEVICE_TIMEOUT = 30004, // 硬件超时
15    DEVICE_MOTIONCONTROLLER_AXIS_ERROR = 30005, // 硬件轴错误
16
17    // 用户引起的错误
18    USER_CANCEL=40001, // 用户取消操作
19
20    // api 判定的错误
21    FUNCTION_NOTIMPLEMENT = 50001, // API 方法还未开放
22    FUNCTION_UNAUTHORIZED = 50002, // API 未授权
23
24    // Others
25    UNKNOWN = 99999 // 未知错误
26 }
```

2.4. 超时时间

涉及控制器硬件位移的方法中，下位机内部会根据位移的速度和距离计算出带有余量的一个预估时间，超时时间仅大于等于该预估时间才生效。

2.5. 通用参数 IParameter

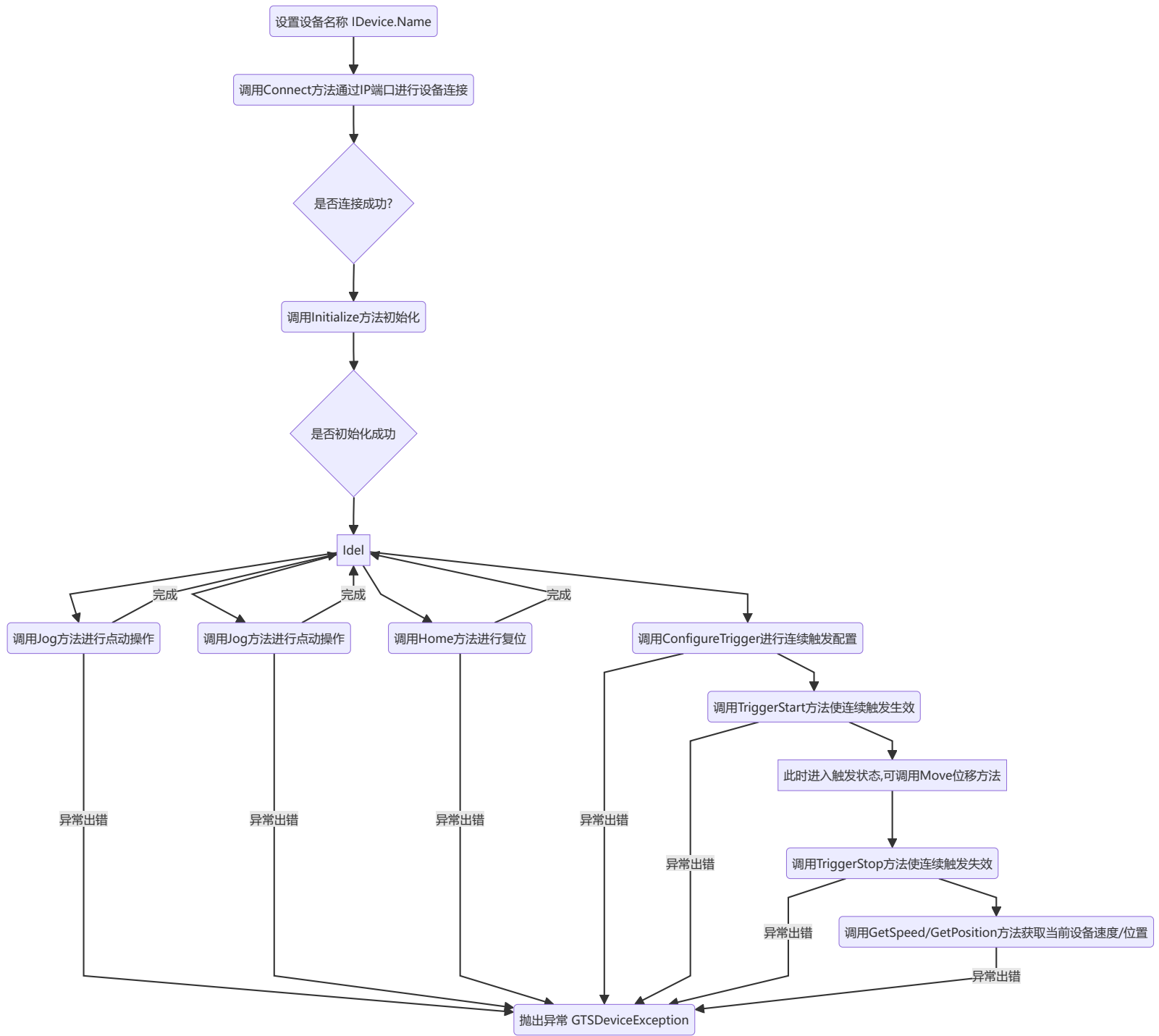
该接口定义在 Platform.Core 中

```
1  /// <summary>
2  /// 数据参数化接口
3  /// </summary>
4  public interface IParameter
5  {
6      /// <summary>
7      /// 获取参数内容
8      /// </summary>
9      /// <typeparam name="T">参数类型</typeparam>
10     /// <param name="key">参数Key</param>
11     /// <param name="value">参数值</param>
12     /// <param name="defaultValue">未获取成功，默认赋参数值</param>
13     /// <returns>是否获取成功</returns>
14     bool TryGet<T>(string key, out T value, T defaultValue = default);
15
16     /// <summary>
17     /// 获取参数内容
18     /// </summary>
19     /// <typeparam name="T">参数类型</typeparam>
20     /// <param name="key">参数Key</param>
21     /// <returns>获取的结果</returns>
22     /// <exception cref="ParameterReadException">读参数异常</exception>
23     T Get<T>(string key);
24
25     /// <summary>
26     /// 设置参数内容
27     /// </summary>
28     /// <typeparam name="T">参数类型</typeparam>
29     /// <param name="key">参数key</param>
30     /// <param name="value">参数值</param>
31     /// <returns>是否设置成功</returns>
32     bool TrySet<T>(string key, T value);
33
34     /// <summary>
35     /// 设置参数内容
36     /// </summary>
37     /// <typeparam name="T">参数类型</typeparam>
38     /// <param name="key">参数key</param>
39     /// <param name="value">参数值</param>
40     /// <exception cref="ParameterWriteException">写参数异常</exception>
41     void Set<T>(string key, T value);
42 }
```

3. 调用流程图

3.1. API方法调用

设备接口方法调用流程如下



4. 获取运动控制器实例

通过运动控制器工厂，传入控制器型号来获取控制器实例对象。

```
1  /// <summary>
2  /// 运动控制器工厂
3  /// </summary>
4  public static class MotionControllerFactory
5  {
6      /// <summary>
7      /// 根据型号获取对应的运动控制器实现
8      /// </summary>
9      /// <param name="modelName">控制器型号</param>
10     /// <param name="args">扩展参数</param>
11     /// <returns>控制器的实现实例</returns>
12     public static IMotionController? GetDeviceInstance(string modelName, params object[] args)
13 }
```

支持的控制器型号如下：

```
1  Zmotion    // 正运动控制器
2  GC         // 高川控制器
```

5. 连接 IConnect

```
1  /// <summary>
2  /// 设备连接接口
3  /// </summary>
4  public interface IConnect
```

5.1. 连接

通过连接地址连接到设备。

```
1  void Connect(string address, IParameter parameters=null);
```

参数

- address: 运动控制器的地址
 - TCP/IP 连接方式 `TCP SOCKET::IP::PORT`
 - 串口连接方式 `SERIAL::PORT::BAUDRATE`
 - 特殊连接方式 `UNIQUE::DRIVERNAME::ADDRESS`
 - lg: `UNIQUE::zmotion::192.x.x.x`, `DRIVERNAME` 为驱动的类型，此处即为控制器型号。
 - 其他: VISA 协议地址

5.2. 断开连接

断开与设备之间的连接。

```
1 | void Disconnect(IPParameter parameters=null);
```

5.3. 当前连接状态

API 需要维护一个当前是否连接的状态

```
1 | bool IsConnected { get; };
```

6. 设备接口 IDevice

```
1 | /// <summary>
2 | /// 设备接口
3 | /// </summary>
4 | public interface IDevice : IConnect, IPParameter
```

6.1. 设备名称

用于流程中对设备进行命名，在抛出异常后能够在异常中知道出错设备的名称，用于提示用户

```
1 | /// <summary>
2 | /// 设备名称
3 | /// </summary>
4 | string Name { get; set; };
```

6.2. 复位

类似于仪表 `*RST` 的做法，将设备的状态完全复位

```
1 | void Reset(IPParameter parameter);
```

参数

- parameter: 扩展参数

6.3. 执行指令

执行动作/指令，预留接口

```
1 | void Execute(IPParameter parameter, IPParameter response);
```

参数

- parameter: 扩展参数
- response: 返回参数

7. 运动控制接口 IMotionController

```
1 | public interface IMotionController : IDevice
```

接口的每个方法中都添加了 `IPParameter parameters` 这一拓展参数，其目的在于避免因为特殊的需求，需要对 API 进行修改时，接口的需要进行变动。如无特殊说明，该参数默认为 null，以下方法中不再另外对 `IPParameter parameter` 做说明。

7.1. 初始化

初始化接口，连接成功以后调用，如果硬件上电后硬件自动初始化，那么 api 直接返回；如果上电后硬件不能自动初始化，api 在该方法中调用驱动层的初始化方法。

```
1 | void Initialize(IPParameter parameters);
```

参数

- parameters: 扩展参数，可传入下面列出的参数：
 - ParameterKey.PulseScales : (必选) 轴当量字典，类型为 Dictionary<int, double>。其中 key: 轴序号，value: 轴当量（轴位移一个单位需要发出的脉冲数量）。轴当量取决于轴接入的电机和实际现场测试环境，请联系电气组获取具体的数值。
- ParameterKey.InitializeFilePath : (可选) 初始化配置文件全路径，类型为 string

7.2. 设备寻零

在连接到设备成功后，对设备参照**硬件物理零点**进行寻零操作。

7.2.1. 同步多轴回零

轴回零，同步方法，当设备达位后方法返回

```
1 | void Home(IEnumerable<int> dimensions, int timeout = -1, IParameter? parameters = null);
```

参数

- dimensions: 需要回零的轴序号。
- timeout: 完成动作的最大允许时间（ms）。请注意，此方法要在内部估计超时的下限，因此，如果上位机提供更小的超时，将抛出GTSDDeviceException错误，错误码为 DEVICE_TIMEOUT。可选参数，如上层程序不设置，则Timeout为设备默认值 -1,-1 表示没有超时时间。

7.2.2. 异步多轴回零

轴回零，异步，当设备轴停止后 Task 返回

```
1 | Task HomeAsync(IEnumerable<int> dimensions, int timeout = -1, IParameter? parameters = null);
```

参数

- dimensions: 需要回零的轴序号。
- timeout: 完成动作的最大允许时间（ms）。请注意，此方法要在内部估计超时的下限，因此，如果上位机提供更小的超时，将抛出GTSDDeviceException错误，错误码为 DEVICE_TIMEOUT。可选参数，如上层程序不设置，则Timeout为设备默认值 -1,-1 表示没有超时时间。
- 返回: 执行 Task, 轴停止后Task返回

7.3. 速度，加速度，减速度设置和读取

7.3.1. 设置速度

对运动速度进行设置

```
1 | void SetSpeed(int dimension,double speed);
```

参数

- dimension : 轴序号
- speed: 速度，单位 Unit/s。

7.3.2. 读取速度

读取当前速度

```
1 | Dictionary<int, double> GetSpeed(IParameter? parameters = null);
```

参数

- return: 从设备获取当前设备位移位置字典信息，若硬件配置存在offset，则附加计算 offset。key-轴序号，value-轴所在位置，单位 Unit

7.3.3. 设置加速度

```
1 | void SetAcceleration(int dimension,double speed);
```

参数

- dimension : 轴序号
- speed: 加速度，单位 Unit

7.3.4. 读取加速度

在连接到设备成功后，获取设备的加速度

```
1 | Dictionary<int, double> GetAcceleration (IPParameter parameters=null);
```

参数

- return 每个轴对应的加速度，key-轴序号，value-加速度，单位 Unit/s/s

7.3.5. 设置减速度

```
1 | void SetDcceleleration(int dimension,double speed);
```

参数

- dimension：轴序号
- speed： 减速度，单位 Unit/s/s

7.3.6. 读取减速度

在连接到设备成功后，获取设备的减速度

```
1 | Dictionary<Dimension, double> GetDcceleleration (IPParameter parameters=null);
```

参数

- parameters : 扩展参数
- return 每个轴对应的减速度，key-轴序号，value-加速度，单位 Unit/s/s

7.4. 设备位移

在连接到设备成功后，通过此方法在合理的超时时间内控制设备位移到指定的角度(位置)。

7.4.1. 同步多轴相对位移

在合理的超时时间内控制设备位移到**相对于本次位置**的指定的角度(位置)，同步方法，当设备达位后方法返回，如中途调用方调用停止方法，本方法将抛出 GTSDDeviceException，ErrorCode = USER_CANCEL

```
1 | void Move(Dictionary<int,double> moveInfoDic ,int timeout = -1,IPParameter parameters=null);
```

参数

- moveInfoDic: 位移参数字典。
 - Dimension: 轴序号。
 - double: 表示设备或轴从**当前位置**需要位移的单位。执行完成时，设备将在提供位置停止 **注意位置正负表示方向。**
- timeout: 完成动作的最大允许时间 (ms)。 请注意，此方法要在内部估计超时的下限，因此，如果上位机提供更小的超时，将将抛出GTSDDeviceException错误，错误码为 DEVICE_TIMEOUT。

7.4.2. 异步多轴相对位移

在合理的超时时间内控制设备位移到**相对于本次位置**的指定的角度(位置)，异步，当设备达位后 Task 返回

```
1 | Task MoveAsync(Dictionary<int, double> moveInfoDic ,int timeout = -1,IPParameter parameters=null);
```

参数

- moveInfoDic: 位移参数字典。
 - Dimension: 轴序号。
 - double: 表示设备或轴从**当前位置**需要位移的单位。执行完成时，设备将在提供的位置停止 **注意位置正负表示方向。**
- timeout: 完成动作的最大允许时间 (ms)。 请注意，此方法要在内部估计超时的下限，因此，如果上位机提供更小的超时，将将抛出GTSDDeviceException错误，错误码为 DEVICE_TIMEOUT。
- 返回： 执行 Task, 轴停止后 Task 返回

7.4.3. 同步多轴绝对位移

在合理的超时时间内控制设备位移到指定的角度(位置)，同步方法，当设备达位后方法返回，如中途调用方调用停止方法，本方法将抛出 GTSDDeviceException，ErrorCode = USER_CANCEL

```
1 | void MoveAbs(Dictionary<int,double> moveInfoDic ,int timeout = -1,IPParameter parameters=null);
```

参数

- moveInfoDic: 位移参数字典。
 - Dimension: 轴序号。

- `double`: 以零点为基准, 设备或轴从**位置原点**需要位移的位置。执行完成时, 设备将在提供的位置停止**注意位置为绝对值**。
- `timeout`: 完成动作的最大允许时间 (ms)。 请注意, 此方法要在内部估计超时的下限, 因此, 如果上位机提供更小的超时, 将抛出GTSDDeviceException错误, 错误码为 DEVICE_TIMEOUT。

7.4.4. 异步多轴绝对位移

在合理的超时时间内控制设备位移到指定的角度(位置), 异步, 当设备达位后 Task 返回

```
1 | Task MoveAbsAsync(Dictionary<int, double> moveInfoDic ,int timeout = -1,IParameter parameters=null);
```

参数

- `moveInfoDic`: 位移参数字典。
 - `Dimension`: 轴的维度。
 - `double`: 以零点为基准, 设备或轴从**位置原点**需要位移的位置。执行完成时, 设备将在提供的位置停止**注意位置为绝对值**。
- `timeout`: 完成动作的最大允许时间 (ms)。 请注意, 此方法要在内部估计超时的下限, 因此, 如果上位机提供更小的超时, 将抛出GTSDDeviceException错误, 错误码为 DEVICE_TIMEOUT。
- 返回: 执行 Task, 轴停止后 Task 返回

7.4.5. 同步点动

匀速连续位移, 点动, 同步方式, 调用后, 设备将以设置的方式持续运动, 直到调用方调用 Stop

```
1 | void Jog(int dimension, bool direction, IParameter? parameters = null);
```

参数

- `dimension`: 轴序号。
- `direction`: 设置设备匀速连续位移的方向, `true`:正向, `false`:反向。

7.4.6. 异步点动

匀速连续位移, 点动, 同步方式, 调用后, 设备将以设置的方式持续运动, 直到调用方调用 Stop

```
1 | Task JogAsync(int dimension, bool direction, IParameter? parameters = null);
```

参数

- `dimension`: 轴序号。
- `direction`: 设置设备匀速连续位移的方向, `true`:正向, `false`:反向。
- 返回: 执行 Task, 在轴停止后返回

7.4.7. 同步停止运动

停止位移, 同步方式, 在轴停止后返回

```
1 | void Stop(IEnumerable<int> dimensions, IParameter? parameters = null);
```

参数

- `dimensions`: 需要停止的轴序号。

7.4.8. 异步停止运动

停止位移, 同步方式, 在轴停止后返回

```
1 | void StopAsync(IEnumerable<int> dimensions, IParameter? parameters = null);
```

参数

- `dimensions`: 需要停止的轴序号。
- 返回: 执行 Task, 在轴停止后返回

7.5. 获取位置信息

```
1 | Dictionary<int, double> GetPosition(IParameter? parameters = null);
```

parameter:

- 返回 :所有轴的当前速度字典。key-轴序号, value-轴速度, 单位 Unit/s

7.6. 获取轴运动状态

```
1 Dictionary<int, bool> GetIsAxisIdle(IParameter? parameters = null);
```

parameter:

- 返回 :所有轴的当前状态字典。key-轴序号，value-轴状态，ture: 当前为 idle 状态，false: 当前为运动状态。

7.7. 清除轴的错误

清除轴的错误，使之能正常运动

```
1 void ClearError(IEnumerable<int>? dimensions = null, IParameter? parameters = null);
```

parameter:

- dimensions :需要清除错误的轴序号，如果为 null，则每个轴的错误都清除。

7.8. 设置原点

将轴当前位置设置为原点位置

```
1 void SetHomePosition(int dimension);
```

parameter:

- dimension :轴序号。

7.9. 设置螺距补偿值

螺距补偿：将运动控制器某轴的指令坐标位置与高精度测量系统所得的实际坐标位置相比较，计算出在全程上的误差，并将该误差以表格的形式输入到控制器中。

其中补偿点位需要在补偿区间内均匀测量后得出误差列表，每个点位都需要同时有正向补偿和负向补偿，补偿数组按**目标位置从小到大**排序。

正向补偿：正向运动到点位后，补偿值 = 目标位置 - 测量位置（实际到达位置）

负向补偿：负向运动到点位后，补偿值 = 测量位置 - 目标位置

```
1 void SetScrewPitchCompensation(int dimension, double startPos, double stopPos, double[] posComp, double[] negComp);
```

参数

- dimension: 轴序号
- startPos: 补偿区间起始位置，单位 Unit
- stopPos: 补偿区间截止位置，单位 Unit
- posComp: 正向运动补偿列表，数量应和 negComp 数量一致。
- negComp: 负向运动补偿列表，数量应和 posComp 数量一致。

7.10. 启用/禁用螺距补偿功能

```
1 void SetScrewPitchCompEnable(int dimension, bool isEnabled);
```

参数

- dimension: 轴序号
- isEnabled: true-启动螺距补偿，false-禁用螺距补偿

8. 脉冲生成发接口 IPulseGenerator

8.1. 连续触发参数配置

配置设备连续触发模式参数，调用后开始进行持续移动，直到上位机调用 Stop 方法。

```
1 void ConfigureTrigger(int dimension, double start, double stop, double step, ushort pulsewidth, bool isRaisingEdge, int timeout = -1,IParameter parameters=null);
```

参数

- dimension: 轴序号。
- start :设置连续触发的开始位置， 单位 Unit。
- stop :设置连续触发的停止位置，单位 Unit。
- step :设置连续触发的步长，单位 Unit。

- `pulsewidth`:连续触发输出的脉冲宽度。脉冲宽度单位 `us`。
- `isRaisingEdge`:脉冲是否为上升沿， `true`: 上升沿脉冲， `false`: 下降沿脉冲
- `timeout`: 完成动作的最大允许时间 (`ms`)。 请注意，此方法将在内部估计超时的下限，因此，如果上位机提供更小的超时，将抛出 `GTSEException` 错误，错误码为 `DEVICE_TIMEOUT`。可选参数，如上层程序不设置，则 `Timeout` 为设备默认值。

8.2. 连续触发开始

启动连续触发模式，启动后，连续触发配置才生效。

```
1 | void StartTrigger(IParameter parameters=null);
```

8.3. 连续触发停止

停止连续触发模式，停止后，连续触发配置不生效。

```
1 | void StopTrigger(IParameter parameters=null);
```

9. 通用输入/输出控制接口 IGpio

对通用单 pin 输入/输出口进行控制

```
1 | /// <summary>  
2 | /// 通用输入/输出控制接口  
3 | /// </summary>  
4 | public interface IGpio
```

9.1. 设置输出口电平

```
1 | void SetGpioOut(int ioIndex, bool value, IParameter? parameters = null);
```

参数

- `ioIndex`: 输出口序号，从 0 开始
- `value`: 输出电平， `true` - 高电平， `false` - 低电平

9.2. 获取输入口电平

```
1 | bool GetGpioIn(int ioIndex, IParameter? parameters = null)
```

参数

- `ioIndex`: 输入口序号，从 0 开始
- 返回: `true` - 高电平， `false` - 低电平